Improving Emscripten Build & Link Performance in Cute Framework

Report date: 2026-07-23 · Analyzed against `origin/master` @ `5c8d0c16`

1. Executive summary

Cute Framework's web (Emscripten) builds are slow to **link** because **ASYNCIFY is enabled globally with no allow/deny list**, and its Binaryen instrumentation pass runs **once per executable — across 54 samples** — during linking. ASYNCIFY rewrites every function that *could* be on the stack when the program yields. Because CF makes pervasive **indirect (function-pointer) calls** — most importantly through the `cf_alloc` allocator hub and the ~8,400-line `cspv` shader compiler — the analysis can't prove those paths never yield, so it instruments almost the whole module. That inflated set is what makes each link slow and each `.wasm` large.

The single-word cause is **indirect calls**, and the fix is to tell ASYNCIFY which of those paths provably never yield, so it stops instrumenting them.

Ranked recommendations

#	Change	Risk	Effort	Expected effect
1	`ASYNCIFY_REMOVE` the allocator hub (`cf_alloc/free/calloc/realloc`)	Low*	Trivial	Large ↓ instrumented set → faster link, smaller/faster wasm
2	`ASYNCIFY_REMOVE=cspv_*` (the shader compiler)	Low	Trivial	Large ↓ (removes ~8.4k-line pure-compute subsystem)
3	Audit/extend the REMOVE list via `-sASYNCIFY_ADVISE` (`yyjson_*`, `cute_shader_*`, …)	Low	Small	Medium ↓, and confidence the list is correct
4	Ninja + parallel linking (`cmake --build build -j`) in CI & locally	None	Trivial	Large ↓ **wall-clock** (54 links run concurrently)
5	Drop DWARF (`-g0`) for CI/release web builds	None	Trivial	Medium ↓ link time & output size
6	Samples-subset target for iteration	None	Small	Large ↓ for day-to-day dev
7	**Strategic:** evaluate JSPI (`-sJSPI`) to replace ASYNCIFY	Medium	Medium–High	Removes the instrumentation pass entirely
8	**Strategic:** precompiled-bytecode-only web build (drop runtime cspv)	Medium	Medium	Removes cspv from the module + from ASYNCIFY
9	**Strategic:** drop `-sASYNCIFY` entirely (remove both `cf_sleep` sites + gate web coroutines)	Medium	High	Eliminates the instrumentation pass — largest possible win

* Low with a documented contract: a custom allocator installed via `cf_allocator_override` must not yield. The default allocator is `malloc`, which trivially satisfies this.

> **Note on numbers.** This environment has no `emsdk`, so this report gives *direction* and a **measurement recipe** (§6) rather than benchmark figures. The changes in §5.1 are low-risk and quick to A/B — measure per-sample link time and `.wasm` size before/after.

---

2. Background: why web builds need ASYNCIFY, and where the cost is

2.1 What ASYNCIFY does and why it's expensive

ASYNCIFY lets synchronous-looking C/C++ code suspend and later resume by unwinding and rewinding the WebAssembly call stack. It is implemented as a whole-module **Binaryen `wasm-opt` transform applied at link time**: every function that might be live on the stack across a suspend point is rewritten to save/restore locals and support unwinding. The cost of that pass — and the size/speed cost in the output — scales with the **number of instrumented functions**. Shrinking that set is the primary lever for link time.

2.2 Where it's configured

`CMakeLists.txt:226-249` (the `cute` INTERFACE link options), propagated to every sample, and re-stated per sample at `samples/CMakeLists.txt:27`:

```
# CMakeLists.txt  (cute INTERFACE link options)
-sASYNCIFY=1
-sASYNCIFY_STACK_SIZE=2040000
# ...no ASYNCIFY_ONLY / ASYNCIFY_REMOVE / allowlist...

# samples/CMakeLists.txt:27  (per sample, at link)
target_link_options(${TARGET_NAME} PRIVATE -o ${TARGET_NAME}.html -sASYNCIFY=1 -O1 -gsepa
```

There are **54 `add_sample()` targets**. Each does its own full ASYNCIFY link, so the expensive pass runs 54 times.

2.3 The actual yield points on web (there are only two kinds)

ASYNCIFY is genuinely required — but only for a small, well-known set of suspend points:

1. **`cf_sleep()` → `SDL_Delay`** (`src/cute_time.cpp:281-283`), called from: - the WebGL fence-wait busy loop, `src/cute_graphics_gles.cpp:426` (`cf_sleep(0)` inside `#ifdef CF_EMSCRIPTEN` — WebGL can't block on `glClientWaitSync`, so it yields to the browser to simulate a GPU wait), and - the frame-rate limiter, `src/cute_time.cpp:106` (`cf_sleep(1)`).
2. **Coroutines** — minicoro (`libraries/edubart/minicoro.h`) uses `MCO_USE_FIBERS` on Emscripten (`emscripten/fiber.h`), which asserts `emscripten_has_asyncify()` at creation and swaps stacks via the ASYNCIFY machinery.

Everything *else* in the module — the renderer's non-blocking paths, math, image decoding, the JSON parser, and the entire shader compiler — **never yields**. It is instrumented only because the analysis can't prove that, thanks to indirect calls.

2.4 The main loop is already a callback — that's not the problem

CF itself never calls `emscripten_set_main_loop`; the samples do, e.g.:

```
#ifdef CF_EMSCRIPTEN
    emscripten_set_main_loop(update, 60, true);   // browser drives update()
#else
    while (app_is_running()) update();
#endif
```

So the *frame loop* is already returning control to the browser each tick and does **not** depend on ASYNCIFY. The remaining ASYNCIFY dependencies are the **intra-frame** `cf_sleep` (fence wait + limiter) and **coroutines** — see §5.3 and §5.5 for how those could be removed to drop ASYNCIFY further.

---

3. Root cause: indirect calls inflate the instrumented set

ASYNCIFY treats an indirect call (`call_indirect`) as *potentially* calling any type-compatible function — including one that yields. Any function that makes such a call is therefore assumed able to yield, and so are all of its callers, transitively. Two hubs dominate CF:

3.1 The allocator hub — `cf_alloc` and friends

`cf_alloc/free/calloc/realloc` dispatch through a **global function-pointer vtable** (`src/cute_alloc.cpp:65-83`, `include/cute_alloc.h`):

```
CF_Allocator s_allocator = s_default_allocator;   // swappable at runtime
void* cf_alloc(size_t size) {
    return s_allocator.alloc_fn ? s_allocator.alloc_fn(size, s_allocator.udata)
                                : s_default_alloc(size, s_allocator.udata);   // indirect
}
```

Essentially **all** CF allocation funnels here: ~180+ direct call sites across `src/`, plus every `Array`/container macro, `CF_NEW`, and the `CK_ALLOC` redirect. Because that one call is indirect, the huge subgraph that allocates is dragged into the instrumented set. The default allocator bottoms out at `malloc` (which never yields), but the function pointer means the toolchain can't *prove* it — so it conservatively instruments everything.

3.2 The shader compiler — `cspv` (`cute_spirv.h`)

`libraries/cute/cute_spirv.h` (**8,415 lines** on `master`) is CF's own GLSL→SPIR-V compiler *plus* the GLSL-ES-300 / HLSL / MSL transpiler backends that used to be SPIRV-Cross. It is compiled straight into the `cute` target (`target_sources(cute PRIVATE cute_shader.cpp)`, `tools/CMakeLists.txt`) and used **at runtime on web** for shader compilation (`CF_RUNTIME_SHADER_COMPILATION`). It is **pure computation and never yields**, yet lands in the ASYNCIFY set because of its own internal indirect calls. Notably, cspv/ckit allocate via **raw** `malloc`, *not* `cf_alloc` — so it is **not** reached through the allocator hub. That is why removing the allocator hub and removing `cspv_*` are **two independent wins**, and both are worth doing.

3.3 Why `ASYNCIFY_IGNORE_INDIRECT` is the wrong tool

`-sASYNCIFY_IGNORE_INDIRECT` tells ASYNCIFY to assume **no** indirect call ever yields. That is unsafe here: the **coroutine API is literally an indirect call to a user function pointer** (`cf_make_coroutine(CF_CoroutineFn* fn, …)` → the fiber runs `fn`, which yields), and the allocator is an indirect call too. Blanket-ignoring indirect calls would drop instrumentation from paths that genuinely suspend, corrupting the stack at runtime. The correct approach is the **targeted allow/deny lists** below, which remove only paths you can prove never yield.

---

4. The safety rule for `ASYNCIFY_REMOVE`

> A function is safe to `ASYNCIFY_REMOVE` **iff it never transitively calls a suspend point** — i.e., never reaches `cf_sleep`, a coroutine yield, or a fiber swap.

Checking the candidates against this rule:

- **`cf_alloc/free/calloc/realloc`** — with the default allocator they call `malloc` / `free` only. `cf_alloc` completes and returns before any yield can occur; it is never *on the stack* at a suspend point. **Safe**, given the documented contract that a custom allocator must not yield.
- **`cspv_*`** — a self-contained compiler; no path to `cf_sleep` or coroutines. **Safe.**
- **`yyjson_*`, `cute_shader_*`, image/math helpers** — pure compute. **Safe** (validate with ADVISE, §5.1).

This rule also explains why removal stays safe even when these functions run *inside* a coroutine: they finish and pop off the stack before the coroutine reaches its `yield`, so they are never live across a suspend.

---

5. Recommendations

5.1 Tier 1 — Shrink the ASYNCIFY instrumented set *(directly targets the cause)*

(1) Remove the allocator hub and (2) the shader compiler. Add an `ASYNCIFY_REMOVE` response file and wire it into the `cute` INTERFACE link options so it applies to every sample:

`cmake/asyncify_remove.txt`

```
# Functions proven to never reach a suspend point (cf_sleep / coroutine yield / fiber swa
# See docs: the safety rule is "never transitively yields".
cf_alloc
cf_free
cf_calloc
cf_realloc
cspv_*
# Candidates to validate with -sASYNCIFY_ADVISE before enabling:
# yyjson_*
# cute_shader_*
```

`CMakeLists.txt` (inside the `if(EMSCRIPTEN)` link options, ~line 244):

```
        -sASYNCIFY=1
        -sASYNCIFY_STACK_SIZE=2040000
        -sASYNCIFY_REMOVE=@${CMAKE_CURRENT_SOURCE_DIR}/cmake/asyncify_remove.txt
```

Why a response file: the list is easy to grow, supports `*` wildcards, and keeps the CMake line readable. (Inline `-sASYNCIFY_REMOVE=cf_alloc,cf_free,...` also works.)

(3) Audit and extend with `-sASYNCIFY_ADVISE`. Build one representative sample with `-sASYNCIFY_ADVISE=1`; Emscripten prints, per function, *why* it is instrumented (which suspending callee, or "indirect call"). Use that to (a) confirm the removals above actually shrank the set, (b) confirm nothing that truly yields was removed, and (c) find the next biggest pure-compute clusters to add. This is the authoritative way to build the list — wildcard patterns match whatever symbol names survive `-O1` (some `static` cspv helpers get inlined into `cspv_compile`/renamed), so verify rather than assume.

Do *not* use `-sASYNCIFY_IGNORE_INDIRECT` (see §3.3).

> **Expected effect:** the allocator hub and cspv are the two largest indirect-call gateways; removing them should collapse a large fraction of the instrumented set, which is exactly what wasm-opt spends its time on. This is the change most worth measuring first.

5.2 Tier 2 — Build orchestration *(independent of ASYNCIFY; biggest wall-clock wins)*

(4) Parallelize the 54 sample links. The Emscripten CI job configures **without Ninja** and builds without an explicit `-j` (`.github/workflows/build.yml`), so sample links are effectively serialized. Switch it (and the local `web.cmd`) to Ninja + parallel build:

```
emcmake cmake -B build -G Ninja -DCMAKE_C_COMPILER_LAUNCHER=ccache -DCMAKE_CXX_COMPILER_L
cmake --build build -j                # or: cmake --build build --parallel
```

The 54 ASYNCIFY links are independent; running them across all cores is a large wall-clock win with zero risk. (ccache already helps *compiles* but does **not** cache the wasm-opt ASYNCIFY link — so parallelism, not caching, is the lever here.)

(5) Drop DWARF debug info for CI/release. Web builds currently keep DWARF (`-gsplit-dwarf` compile, `-gseparate-dwarf` sample link). Debug info adds link time and output size. Emit it only in a debug preset; use `-g0` for CI/release:

```
# release/CI web: -g0   ·   debug preset: keep -gseparate-dwarf
```

(6) A samples-subset target for iteration. Building all 54 samples on every change is the dominant local cost. Add `EXCLUDE_FROM_ALL` to the sample executables plus a small curated `web_smoke` target (a handful of representative samples), so day-to-day iteration relinks a few, not 54. CI can still build the full set on demand.

5.3 Tier 3 — Strategic options *(larger, higher-payoff)*

(7) Evaluate JSPI (`-sJSPI`, formerly `ASYNCIFY=2`). JavaScript Promise Integration performs stack switching in the **VM** instead of via Binaryen instrumentation. It **eliminates the wasm-opt ASYNCIFY pass**, so link time drops sharply and the output is smaller/faster. Caveats to validate before committing: - **Browser support**: shipping in Chromium (137+, on by default in 2025); Firefox/Safari were still behind flags / in progress as of early 2026 — so a fallback build may be needed. - **Coroutines**: minicoro's

Emscripten path uses classic `emscripten_fiber_*` (ASYNCIFY). JSPI compatibility with fibers must be verified, or the coroutine strategy migrated. - **Sleep model**: JSPI suspends on a Promise-returning import, so `cf_sleep`'s fence-wait usage would need to be expressed as an async import. Recommend a spike behind a CMake option (`CF_WEB_JSPI`) to measure link time and validate coroutines + the GLES fence path.

(8) Precompiled-bytecode-only web build. cspv is linked into web binaries purely to compile shaders (including built-ins) at runtime. If web builds consumed only precompiled SPIR-V via `cf_make_shader_from_bytecode` (the `cute-shaderc` offline path already exists), cspv could be *excluded from the module entirely* on web — removing it from both the binary and the ASYNCIFY set, and shrinking download size. Trade-off: runtime shader authoring from source would be unavailable on web. A build option could gate this.

(9) Drop `-sASYNCIFY` entirely by removing the two intra-frame suspend points. This is the highest-payoff option: with no ASYNCIFY at all, the Binaryen instrumentation pass never runs — links get dramatically faster and the `.wasm` is smaller and faster. The main loop is *already* callback-driven (`emscripten_set_main_loop`), so the loop itself does not need ASYNCIFY. Only two things do:

- **The WebGL fence busy-wait** (`cf_sleep(0)`, `src/cute_graphics_gles.cpp:426`) — WebGL can't block on `glClientWaitSync`, so the GLES backend yields mid-frame to fake a GPU wait. Fix: restructure the ring buffer to **poll the fence across `requestAnimationFrame` ticks** — if the slot about to be reused isn't signaled, return from `update()` and re-check next tick instead of sleeping. Because the loop is already a browser callback, this is a natural fit.
- **The frame limiter** (`cf_sleep(1)`, `src/cute_time.cpp:106`) — redundant on web, since `emscripten_set_main_loop`/rAF already paces frames. `#ifdef CF_EMSCRIPTEN`-compile it out.
- **Coroutines** — minicoro's Emscripten fibers hard-assert `emscripten_has_asyncify()`, so any app using `cf_make_coroutine` needs ASYNCIFY. Dropping ASYNCIFY therefore has to gate out coroutines on web.

Deliver this behind a build option (e.g. `CF_WEB_NO_ASYNCIFY`) that selects the non-blocking fence path, compiles out the web limiter sleep, and disables web coroutines. Larger refactor than Tier 1, but it removes the cost at the source rather than shrinking it.

> **On `emscripten_set_main_loop_arg` specifically:** it does *not* by itself allow dropping ASYNCIFY. It is just `emscripten_set_main_loop` plus a `void* udata` argument (cleaner than globals). The loop is already a callback and already ASYNCIFY-independent; the two `cf_sleep` sites above are what keep `-sASYNCIFY` mandatory. Switching to the `_arg` variant is a fine, compatible cleanup, but the lever is removing those suspend points — not the loop API.

Minor: `-sMALLOC=emmalloc` (smaller allocator), and revisit `-sASYNCIFY_STACK_SIZE=2040000` (~2 MB) once the instrumented set shrinks — it is runtime memory, not link time, but worth right-sizing.

---

6. Verification methodology

Because build machines differ, measure locally with an `emsdk` install:

1. **Baseline.** Clean build all samples, capturing per-target link time and `.wasm` size: `bash emcmake cmake -B build_web -G Ninja -DCF_FRAMEWORK_STATIC=ON time cmake --build build_web -j`

`ls -l build_web/*.wasm # record sizes`

2. **Instrumented-set audit.** Rebuild one sample with `-sASYNCIFY_ADVISE=1` and save the report; count instrumented functions.
3. **Apply Tier 1** (the `ASYNCIFY_REMOVE` list) and repeat 1–2. Compare: instrumented function count, total link wall-clock, and `.wasm` sizes.
4. **Correctness smoke test.** Run in a browser: a coroutine-using sample and a GPU-bound sample (exercises the fence-wait). Confirm no hang/crash and correct behavior — this validates that nothing which truly yields was removed.
5. **Apply Tier 2** (Ninja `-j`, `-g0`) and re-time the full build for wall-clock.

Report the deltas from steps 3 and 5 as the headline results.

---

7. Appendix

7.1 Proof-of-concept patch (Tier 1)

- New file `cmake/asyncify_remove.txt` (contents in §5.1).
- One added line in `CMakeLists.txt` inside the `if(EMSCRIPTEN)` block: `-sASYNCIFY_REMOVE=@${CMAKE_CURRENT_SOURCE_DIR}/cmake/asyncify_remove.txt`.

This POC is intentionally the lowest-risk subset (allocator hub + cspv). Extend via ADVISE (§5.1 step 3) once measured.

7.2 Key file references (`origin/master`)

Concern	Location
Web ASYNCIFY config	`CMakeLists.txt:226-249`
Per-sample web link	`samples/CMakeLists.txt:24-28` (54 `add_sample` calls)
Allocator indirection	`src/cute_alloc.cpp:65-83`, `include/cute_alloc.h`
`cf_sleep` → `SDL_Delay`	`src/cute_time.cpp:281-283`
Fence-wait yield	`src/cute_graphics_gles.cpp:411-427` (`cf_sleep(0)`)
Frame limiter yield	`src/cute_time.cpp:106` (`cf_sleep(1)`)
Coroutines (fibers)	`src/cute_coroutine.cpp`, `libraries/edubart/minicoro.h` (`MCO_USE_FIBERS`)
Shader compiler (cspv)	`libraries/cute/cute_spirv.h` (8,415 lines), `tools/CMakeLists.txt`, `tools/cute_shader.cpp`
Emscripten CI job	`.github/workflows/build.yml`

7.3 Evaluation of the originally proposed ideas

Idea	Verdict
`-sASYNCIFY_REMOVE=cspv_*` to drop the shader compiler	✅ **Do it.** Pure-compute, never yields; now larger and the sole runtime shader compiler → bigger win than before.
Add `cf_alloc` (and friends) to `ASYNCIFY_REMOVE`	✅ **Do it**, with a documented "custom allocators must not yield" contract. Severs the CF-wide indirect allocation hub — the highest-leverage single change.
"Only ignore everything in `cspv_*`; its one external indirect call is the allocator"	⚠️ **Refine.** cspv uses raw `malloc`, not `cf_alloc`, so it is *not* reached via the allocator hub — it's instrumented by its own internal indirect calls. Removing `cspv_*` is still correct and valuable; just note the two removals are independent.
`-sASYNCIFY_IGNORE_INDIRECT`	❌ **Unsafe** — the coroutine API and allocator are indirect calls that genuinely yield. Use targeted `ASYNCIFY_REMOVE` instead.
"Callback way of handling the main loop"	ℹ️ Already done — samples drive the loop via `emscripten_set_main_loop`. ASYNCIFY's remaining need is the *intra-frame* `cf_sleep` fence wait and coroutines, not the loop (see §5.3-9).
`emscripten_set_main_loop_arg` to drop ASYNCIFY	⚠️ **Won't drop it alone.** The `_arg` variant just adds a `void* udata`; the loop is already ASYNCIFY-independent. Dropping `-sASYNCIFY` requires removing the two `cf_sleep` sites and gating web coroutines (see §5.3-9). A worthwhile cleanup, but not the lever.